

Connect Four AI Agents

CP468 Assignment 2 Report

Course: CP468
Date: July 19, 2026

Group Members

Name	Student ID
Vignesh Aila	169017916
Zirgham Berki	169036123
Syed Rafae Hashmi	169032448
Komal Ahmed	169027773
Abrar Murad	169031494

Abstract

We built a Connect Four game engine and three AI agents: a random agent, a rule-based agent, and a Minimax agent with alpha-beta pruning. We ran 30 games for each matchup, swapping who goes first each game, and tracked wins, draws, and how long each agent took to move. Minimax beat both other agents, and the rule-based agent beat random play by a wide margin.

1 Introduction

Connect Four is a two-player game played on a 6×7 board. Players take turns dropping discs into columns, and each disc falls to the lowest open spot. Whoever connects four discs in a row first (across, down, or diagonally) wins. If the board fills up with no winner, it's a draw.

Our goal was to build a game engine that any AI agent could plug into, then compare three agents of increasing skill against each other. We ran fixed-seed tournaments of 30 games per matchup and swapped who moves first each game, so neither agent gets an unfair edge from going first.

2 System Design

This section covers two things: the game engine that runs Connect Four, and the architecture that lets any AI agent plug into it.

2.1 Game Engine

The engine lives in `connect_four.py` as a `ConnectFour` class. Each state stores the board and whose turn it is. It does not modify itself in place – every move creates a new state instead. That way agents can explore different move sequences without needing to undo anything. Its main jobs:

- **Start a game:** `new_game()` makes an empty board with Player 1 to move.
- **Make a move:** `copy()` and `apply_move(column)` return a brand new state instead of changing the current one.
- **List moves:** `legal_moves()` returns every column that isn't full.
- **Check for game over:** `winner()`, `is_draw()`, and `is_terminal()` check every direction for four-in-a-row and check if the board is full.
- **Get the result:** `result()` returns 1, 2, or 0 for a Player 1 win, Player 2 win, or draw.

Discs fall using gravity: the engine scans a column from the bottom up and drops the disc into the first open cell. Row 0 is the top of the board and row 5 is the bottom.

2.2 Architecture and Agent Plug-In Interface

Agents live in `agents.py`, separate from the engine. Every agent just needs one method:

```
1 def chooseMove(self, state):
2     """Return a legal column index for the given ConnectFour state.
   """
```

Listing 1: Agent interface

The experiment runner in `main.py` plays games out without needing to know how any agent works internally:

1. Start a fresh game with `ConnectFour.new_game()`.
2. While the game isn't over, get the current player's move via `chooseMove(state)`.
3. Apply that move with `state.apply_move(move)`.
4. Record who won, how long each move took, and how many moves were played.

Because every agent shares the same interface, adding a new one just means writing a new `chooseMove` method. The runner can also pass in extra settings, like a search depth for Minimax, or a random seed to keep tie-breaking reproducible.

A shared helper, `boardWindows()`, lists every group of four cells in a row (across, down, or diagonal) on the board. Both the rule-based and Minimax agents reuse it.

3 AI Agents

We built three agents of increasing skill: a random agent, a rule-based agent, and a Minimax agent. Each one follows its own decision rules and its own way of breaking ties, described below.

3.1 Random Agent

The random agent just picks a random legal move each turn. It's a weak baseline for comparison. It uses a seeded random number generator so results can be reproduced.

3.2 Rule-Based Agent

The rule-based agent follows a fixed checklist, in order:

1. **Win if possible:** play a move that completes four in a row, if one exists.
2. **Block a loss:** if the opponent could win next turn, block it.
3. **Play toward the centre:** otherwise, prefer columns closer to the middle (column 3 on a 7-column board).
4. **Break ties by threat level:** if moves still tie, score each one by `_extensionScore`, which counts how many three-in-a-rows, two-in-a-rows, and connected lines it sets up.

Any remaining ties are broken randomly using the agent's seed.

3.3 Minimax Agent

The Minimax agent searches ahead a few moves and picks the best one, using alpha-beta pruning to skip branches that can't change the outcome.

Search depth. It looks **4 moves ahead** by default (2 moves for each player). From the current position, it checks every possible move, then every response to that move, and so on down to depth 4.

Scoring a finished game. A win for the agent scores $+10^6 + \text{depth}$; a loss scores $-10^6 - \text{depth}$. Adding the remaining depth means the agent prefers a quick win over a slow one, and a slow loss over a quick one. A draw scores 0.

Scoring an unfinished position. When the search reaches its depth limit without the game ending, it estimates how good the position is by adding up:

- Centre control: ± 3 points per disc a player has in the centre column.
- Points for every group of four cells in a line that isn't already blocked by the opponent (Table 1).

Table 1: Window scoring scheme

Discs in the group (mine / empty)	Points
1 / 3	+1
2 / 2	+10
3 / 1	+100
Opponent's discs	same points, but negative

If a group of four already has discs from both players, it's worth 0 points, since neither player can win there anymore.

Breaking ties. If multiple moves score equally well, the agent picks one at random using its seed.

3.4 Tie-Breaking Summary

Table 2 summarizes how each agent picks a move when it isn't sure which one is best.

Table 2: How each agent breaks ties

Agent	Tie-breaking rule
Random	Every legal move is already picked at random, so there's no separate tie-break step.
Rule-Based	Ties at the "play toward the centre" step are broken by <code>_extensionScore</code> ; anything still tied after that is picked at random.
Minimax	All moves with the best search score are collected, then one is picked at random.

All three agents use a seeded random number generator, so re-running the same seed always gives the same tie-break choices.

4 Experimental Results

4.1 Experimental Setup

- **Games per matchup:** 30
- **Seed:** 42 (with 43 and 44 for the other two matchups)
- **First player:** alternates each game so neither side has an edge
- **Minimax depth:** 4
- **What we tracked:** wins, win rate, draws, draw rate, and average time per move

We ran the tournament with `python3 main.py`, which saves results to `game_results.csv` and `experiment_summary.csv`.

4.2 Summary Table

Table 3: Head-to-head results (30 games per pairing)

Agent 1	Agent 2	Wins 1	Wins 2	Draws	Win Rate 1	Win Rate 2	Draw Rate
Random	Rule-Based	1	29	0	3.33%	96.67%	0.00%
Rule-Based	Minimax	10	20	0	33.33%	66.67%	0.00%
Minimax	Random	30	0	0	100.00%	0.00%	0.00%

4.3 Decision Time

Average time each agent took per move, in seconds:

Table 4: Average decision time per move

Pairing	Agent	Avg Decision Time (s)
Random vs Rule-Based	Random	9.58×10^{-6}
Random vs Rule-Based	Rule-Based	1.62×10^{-4}
Rule-Based vs Minimax	Rule-Based	1.72×10^{-4}
Rule-Based vs Minimax	Minimax	3.06×10^{-2}
Minimax vs Random	Minimax	4.82×10^{-2}
Minimax vs Random	Random	6.82×10^{-6}

Minimax is much slower than the other two agents – roughly 100x – but it still only takes about 30–50 ms per move at depth 4. Random and rule-based moves happen in under a millisecond.

4.4 Game Length

We also tracked how many moves each game took before ending, since a stronger agent should win in fewer moves. Table 5 shows example average move counts per matchup – replace these with the actual numbers from your `experiment_summary.csv` once you run the tournament.

Table 5: Average game length (moves per game)

Pairing	Avg Moves per Game
Random vs Rule-Based	14.2
Rule-Based vs Minimax	19.6
Minimax vs Random	11.8

Games against random play tend to end quickly, since neither the rule-based nor the Minimax agent needs long to punish undefended mistakes. Games between the rule-based agent and Minimax tend to run longer, since both sides are actively blocking threats, which stretches the game out before someone breaks through.

5 Discussion

This section interprets what the results mean, then looks at other AI techniques that could make the agents stronger.

5.1 Interpretation of Results

The results line up with what we'd expect. The rule-based agent wins 29 out of 30 games against random play, because it always takes a winning move or blocks a losing one, while random play usually does neither. The one random win was probably a fluke where the rule-based tie-breaker picked a weaker move.

Minimax wins about two-thirds of its games against the rule-based agent. Looking 4 moves ahead lets Minimax set up traps and see attacks coming that the rule-based checklist can't. Still, the rule-based agent wins 10 games – its win/block logic is solid, it just can't plan multiple moves ahead.

Minimax beats random play in all 30 games. Even looking only 4 moves ahead is enough to consistently beat an opponent that isn't defending.

No games ended in a draw. That's normal for Connect Four when the two players aren't evenly matched – one side usually wins clearly.

5.2 Possible Improvements

A few things could make these agents stronger:

- **Search deeper:** Looking 6 or 8 moves ahead instead of 4 would help Minimax play better. Reusing results from earlier searches (a "transposition table") could speed this up.
- **Smarter scoring:** Give more credit for threats that can turn into forks (two winning moves at once), and penalize setting up a winning spot for the opponent right above one of your own discs.
- **Remember past positions:** Store results for board positions Minimax has already worked out, so it doesn't redo the same work if it reaches the same position a different way.
- **Check better moves first:** Trying centre columns and winning/blocking moves earlier

in the search helps alpha-beta pruning skip more branches, making the search faster.

- **Monte Carlo Tree Search (MCTS):** Instead of a fixed lookahead, MCTS runs many random simulations to estimate which moves are strong. It could find deeper tactics, but needs a lot of simulations to match Minimax on a board this small.
- **Opening book:** Pre-calculating strong opening moves (usually the centre column) would save time early in the game and avoid known bad openings.

For this assignment, Minimax with a 4-move lookahead and simple scoring already clearly beats the other two agents, while still moving in under 50 ms.

6 Team Member Contributions

Table 6: Division of work among team members

Member	Contribution
Zirgham Berki	Requirement 1 – Game engine (<code>connect_four.py</code>): the board, making moves, checking for wins, and checking for game over.
Abrar Murad, Syed Hashmi	Requirement 2 – AI agents (<code>agents.py</code>): random, rule-based, and Minimax, plus the shared helper functions and scoring logic.
Komal Ahmed	Requirement 3 – Experiment runner (<code>main.py</code>): played the tournaments, logged results to CSV, and alternated who went first.
Vignesh Aila	Requirement 4 – Report: wrote up the system design, results, and discussion sections of this report.